

Praxis

*A reading community organized by depth of engagement, not metrics of attention.
Yumi is the companion inside it.*

What this document is

A printable companion for the early build of Praxis. Two parts: Part One holds the ready-to-paste agent prompts for Stage 0 (writing the Yumi voice document) and Stage 1 (the code foundation). Part Two walks through the four external accounts the project needs: Bookshop.org, GitHub, Firebase, and Netlify.

How to use it

Read the cover, glance at the order-of-operations cheat sheet on the next page, then work through the document in the order it lays out. The setup work is split across the build, not front-loaded. Bookshop, GitHub, and Firebase happen before Stage 0. Netlify happens after Stage 0, before Stage 1. This avoids deploying an empty repo.

Real order of operations: Bookshop application (apply, then wait) → GitHub repo → Firebase setup → Stage 0 (voice document) → Netlify site → Stage 1 (foundation code). The cheat sheet on the next page lays this out visually.

The actual sequence

From nothing to working app. Each step has its full instructions later in this document — this page is the one-glance overview.

1	Apply for Bookshop.org affiliate	External — approval takes days. Apply early, do everything else while waiting.
2	Create the GitHub repo	Empty repo. Push an initial commit with just your existing markdown files (spec, timeline, b
3	Set up Firebase	Project, Authentication (Google), Firestore in test mode. Save the config object somewhere
4	Stage 0 — Yumi voice document	Fresh Claude chat. Produces docs/yumi-voice.md. Push to GitHub when done.
5	Set up Netlify	Connect to the GitHub repo. Site deploys what's there. Set ANTHROPIC_API_KEY env va
6	Stage 1 — Foundation	Claude Code. Five sub-stages. Repo skeleton, state, Firebase wiring, ISBN fetch, PWA sh

Three things to know

Bookshop is the slow external dependency. Their approval can take a week. Apply on day one even if you do nothing else for a few days.

You can do steps 1, 2, and 3 in parallel — they don't depend on each other. Get all three started, then move to Stage 0.

Netlify happens after Stage 0, not before. No point in deploying an empty repo. Once Stage 0 produces yumi-voice.md, the repo has at least one real artifact worth deploying.

Contents

ORDER OF OPERATIONS

The actual sequence

PART ONE — STAGES

- 01 Stage 0 — Yumi voice document
- 02 Stage 1 — Foundation

PART TWO — SETUP STEPS

- 03 Pre-flight checklist
- 04 Bookshop.org setup (apply early)
- 05 GitHub setup
- 06 Firebase setup
- 07 Netlify setup (after Stage 0)

PART THREE — AFTER EACH STAGE

- 08 What to save, log, and verify

Part One

Stages — the prompts you paste to the agent

Yumi voice document

A writing task, not a code task. Produces one markdown file at **docs/yumi-voice.md** that defines who Yumi is, what she does, what she does not do, how she sounds, and what she redirects. Every later stage of the build depends on this document.

How to run this stage

Open a fresh **Claude chat on the web or desktop app** (not Claude Code — this is a writing task). Upload three files: **book-app-spec.md**, **01-stage-0-voice-document.md**, and your HQ files **yumi-brain.js** and **yumi-ui.js** for convention reference. Paste the prompt block below. The agent runs pre-flight, then walks you through three sub-stages: draft, refine, finalize.

Estimated time

About one week at 20 hours per week is the rough guideline. Stage 0 is writing work, not code work, and it runs slower-feeling than the rest of the build — sitting with bad drafts, re-reading aloud, deleting things that sound generic. Three days is normal. Two weeks is also normal. Do not rush this — voice quality is the single most important deliverable for the entire app's brand.

The prompt — copy everything between the lines

PROJECT: Praxis — standalone reading community app, Goodreads-class scope, brand position grounded in critical pedagogy. Greenfield build. Patterns mirror Preston's HQ but the book app is its own codebase.

STAGE: Stage 0 — Yumi Voice Document

GOAL: A canonical markdown document at /docs/yumi-voice.md that defines who Yumi is, what she does, what she does not do, how she sounds, and what she redirects. The document is detailed enough that a stranger could read it and understand Yumi well enough to write in her voice. Every later stage of the build depends on this document.

=====

EXECUTION MODE: Staged build with one human checkpoint per sub-stage.

=====

This is a **WRITING** task, not a code task. Sub-stages are draft / refine / finalize. Self-checks are about content completeness and quality, not byte deltas or syntax.

At the end of each sub-stage:

1. Show the current draft of the document
2. Output a CHECKPOINT REPORT listing each self-check as PASS/FAIL
3. Then either:
 - (a) Ask ONE specific question if genuine ambiguity blocks the next sub-stage, or
 - (b) Output "Sub-stage [0.M] complete — ready to proceed to Sub-stage [0.M+1]?"
4. Wait for explicit GO before continuing

Never bundle sub-stages. Never ask more than one question per checkpoint. Never accept "I covered X" without showing the section that covers it.

=====

PRE-FLIGHT – read these before writing any draft.

=====

The agent must read these files and confirm what's in them before drafting:

1. /home/preston/HQ-DEPLOY/js/yumi-brain.js

Look for: existing system prompt patterns, redirect language, persona references. The book-app Yumi shares personality with HQ Yumi – same character, fresh memory.

2. /home/preston/HQ-DEPLOY/js/yumi-ui.js

Look for: how Yumi's chat UI presents her (tone of empty states, send button labels, error messages). Voice extends to UI copy.

3. The Yumi sections of the spec – §5 and §9 of book-app-spec.md

Look for: the brand-position constraints, the redirect language ("I'd rather read it with you"), the pedagogical scaffolding for Book Artifacts (Freire, hooks, Giroux, De Lissovoy).

Pre-flight output:

- "Confirmed: HQ Yumi system prompt at yumi-brain.js line [N], excerpt: [first 100 chars]"
- "Confirmed: redirect language pattern in [file] at [line]"
- "Confirmed: spec §5 names Yumi memory scope as per-user, fresh on signup"
- "Up to 3 questions if genuinely ambiguous: [questions]"
- "Pre-flight complete – proceeding to Sub-stage 0.1?" if all confirmed

If the HQ files are not accessible, the agent should ask before proceeding. Do not invent HQ patterns.

=====

NON-GOALS – what Stage 0 must NOT do.

=====

- Do NOT write code. Stage 0 produces one markdown file. Nothing else.
- Do NOT modify HQ files. Read-only reference only.
- Do NOT define implementation details (which model, what max tokens, what proxy URL). Those go in Stage 2 when yumi-brain.js is built.
- Do NOT define UI copy beyond a few representative phrases. Full UI copy comes in later stages.
- Do NOT shorten or simplify the brand position. Yumi never summarizes books – this is non-negotiable and must appear explicitly in the document.

=====

CONVENTIONS – for the document itself.

=====

- File path: /home/preston/praxis-app/docs/yumi-voice.md
- Format: markdown
- Length: 800 to 1500 words (long enough to be specific, short enough to actually load at runtime)
- Voice of the document itself: clear, declarative, warm, in the same register Yumi speaks in. The document is a model of her voice as well as a description of it.
- No tables. No bullet points where prose works better. Sections separated by markdown headings.

=====

SUB-STAGES.

=====

Sub-stage 0.1 – Draft

What it produces:

- Initial draft of /home/preston/praxis-app/docs/yumi-voice.md
- Six sections, in this order:
 1. Who Yumi is (her stance, her tradition, her warmth)
 2. What she does (the work she's actually doing for the user)
 3. What she does not do (the explicit refusals – most importantly, never summarizes books)
 4. How she sounds (concrete examples of her voice – three to five sample phrases or exchanges)
 5. The redirects (what she says when asked to do things she doesn't do)
 6. How she changes across contexts (book conversations vs. arc conversations vs. weekly prompts)

Self-checks for Sub-stage 0.1:

- File exists at /home/preston/praxis-app/docs/yumi-voice.md (PASS if `ls` finds it)
- File has all six sections present, each with a markdown heading (PASS if grep finds six h2 headings)
- Section 3 contains the phrase "never summarize" or equivalent explicit refusal language
- Section 5 contains the canonical redirect: "I'd rather read it with you. Where are you in it?"
- File is between 800 and 1500 words (PASS if `wc -w` returns within range)
- File reads as markdown without syntax errors (no broken headings, no unclosed code fences)

Verification commands:

- `ls -la /home/preston/praxis-app/docs/yumi-voice.md`
- `grep -c "^## " /home/preston/praxis-app/docs/yumi-voice.md`
- `grep -c "never summarize\|don't summarize\|won't summarize" /home/preston/praxis-app/docs/yumi-voice.md`
- `grep -c "rather read it with you" /home/preston/praxis-app/docs/yumi-voice.md`
- `wc -w /home/preston/praxis-app/docs/yumi-voice.md`

After all checks PASS, show the full document text in the checkpoint report so Preston can read it before approving.

Sub-stage 0.2 – Refine

What it produces:

- Revised /home/preston/praxis-app/docs/yumi-voice.md based on Preston's feedback after 0.1
- All Preston's edit requests applied
- Voice tightened where it sounds generic or AI-flavored

Self-checks for Sub-stage 0.2:

- All Sub-stage 0.1 checks still PASS
- Every edit Preston requested is reflected, confirmed by quoting the changed line in the checkpoint report
- No section was deleted or shortened beyond what Preston asked for
- The phrase "I'd rather read it with you. Where are you in it?" still appears in Section 5
- The "never summarize" constraint still appears in Section 3

If Preston's edits would remove or weaken the brand position constraints (the never-summarize rule, the privacy of the notebook, the one-artifact-per-book rule), STOP and surface the conflict. Do not silently weaken constraints that the spec marks non-negotiable.

Sub-stage 0.3 – Finalize

What it produces:

- Final /home/preston/praxis-app/docs/yumi-voice.md
- A second file: /home/preston/praxis-app/docs/yumi-voice-changelog.md with date, version 1.0.0, and a one-paragraph summary of what the document covers

Self-checks for Sub-stage 0.3:

- All earlier checks still PASS
- yumi-voice-changelog.md exists with version 1.0.0 entry
- Final document re-read end to end with no editorial concerns

=====

FINAL SUMMARY – at end of Sub-stage 0.3 only.

=====

...
STAGE 0 COMPLETE

Files created:

- /home/preston/praxis-app/docs/yumi-voice.md ([N] words, [N] bytes)
- /home/preston/praxis-app/docs/yumi-voice-changelog.md ([N] bytes)

Sections in voice document: 6 / 6 expected

Brand position constraints verified: yes (never-summarize present, redirect present)

Document register: matches Yumi's voice (per Preston's review)

Open questions for Preston: [list or "none"]

Recommended next stage: Stage 1 – Foundation (state.js, integrations.js, Firebase, auth)

...
=====

NOTES FOR THE AGENT.

=====

This document is the spine of every later interaction Yumi has with users. If you find yourself writing in a register that feels generic, AI-customer-service-bot, or overly cheerful, stop and rewrite. Yumi is warm but serious. She has read deeply. She does not perform enthusiasm. She does not say "Great question!" She does not flatter the user. She asks better questions than she answers. She sounds like a thoughtful friend who has read the same books and remembers them.

If a section feels short, lean into specificity. Examples beat abstractions. A single concrete redirect ("I'd rather read it with you. Where are you in it?") teaches more than three paragraphs about what Yumi avoids.

Length matters. This document is loaded at runtime as part of Yumi's system prompt. Too long and it bloats every conversation; too short and her voice drifts. Aim for 1000 to 1200 words.

What you do during Stage 0: read each draft as it appears. Push back on anything that sounds AI-generic, overly cheerful, or like customer service. The phrase 'great question!' appearing anywhere in the document is a fail. Yumi sounds like a thoughtful friend, not a chatbot.

Foundation

The first real code stage. At the end: repo structure exists, Google sign-in works, ISBN fetch works, the app installs as a PWA. No user-facing UI yet — this is plumbing.

How to run this stage

Open **Claude Code** in your project folder. Make sure your HQ-DEPLOY folder is accessible to the agent so it can read your existing patterns. Paste the prompt block below. The agent runs pre-flight, then walks you through five sub-stages.

Preconditions

- Stage 0 complete — yumi-voice.md saved at docs/yumi-voice.md
- GitHub repo created and project folder initialized as git repo
- Firebase project created with Authentication and Firestore
- Firebase config object saved (you will paste it during sub-stage 1.3)
- Netlify site connected to GitHub repo

Estimated time

Two to three weeks at 20 hours per week. The Firebase config request in sub-stage 1.3 is the only thing that can stall you mid-stage if accounts are not ready.

Sub-stages of Stage 1

1.1	Repo skeleton	13 new files, fixed load order, console logs init
1.2	State schema	SCHEMA_VERSION, ls/sv wrappers, v1 collections
1.3	Firebase + auth	Google sign-in works, persists across refresh
1.4	Book fetch	Open Library + Google Books, ISBN returns valid book
1.5	PWA shell	Service worker registers, app installable

The prompt — copy everything between the lines

PROJECT: Praxis – standalone reading community app, Goodreads-class scope, brand position grounded in critical pedagogy. Greenfield build. Patterns mirror Preston's HQ but the book app is its own codebase.

STAGE: Stage 1 – Foundation

GOAL: A working app skeleton where:

- The repo structure exists at /home/preston/praxis-app/ with the seven-file modular pattern
- state.js carries SCHEMA_VERSION, ls/sv localStorage wrappers, and a stub schema for users/books/userBooks/notebooks
- integrations.js carries Firebase init, Google Auth init, Open Library fetch, Google Books fetch, Netlify-proxy URL constant
- A user can sign in with Google and the auth state persists to localStorage
- The app can fetch a book by ISBN from Open Library and display its title and author in the console
- state syncs minimal user data to Firebase on sign-in
- index.html loads the seven files in fixed order with stubs for those that come later (yumi-brain, arcs, yumi-ui, views, app)

PRECONDITION: Stage 0 is complete. /home/preston/praxis-app/docs/yumi-voice.md exists.

=====

EXECUTION MODE: Staged build with one human checkpoint per sub-stage.

=====

At the end of each sub-stage:

1. Run all self-checks defined for that sub-stage
2. Output a CHECKPOINT REPORT with each check as PASS/FAIL plus the actual evidence (grep output, byte counts, error messages)
3. Then either:
 - (a) Ask ONE specific question if genuine ambiguity blocks the next sub-stage, or
 - (b) Output "Sub-stage [1.M] complete – ready to proceed to Sub-stage [1.M+1]?"
4. Wait for explicit GO before continuing

Never bundle sub-stages. Never ask more than one question per checkpoint. Never accept "I created X" without showing the file size and grep evidence.

=====

PRE-FLIGHT – read these before writing any new code.

=====

The agent must read these HQ files and confirm conventions before writing the book app:

1. /home/preston/HQ-DEPLOY/js/state.js

Confirm: ls(k, d) and sv(k, v) function signatures. Report exact lines.

Confirm: SCHEMA_VERSION constant pattern (if present).

2. /home/preston/HQ-DEPLOY/js/integrations.js

Confirm: how Firebase is initialized.

Confirm: how the Netlify proxy URL is referenced (constant name and value).

3. /home/preston/HQ-DEPLOY/netlify/functions/claude-proxy.js

Confirm: the function signature and the body schema it expects.

4. /home/preston/HQ-DEPLOY/index.html

Confirm: the fixed file load order.

Confirm: the meta and link tags pattern (PWA manifest, service worker registration).

5. /home/preston/HQ-DEPLOY/sw.js (if it exists, otherwise note it's missing)

Confirm: the cache versioning pattern (CACHE_VERSION).

Pre-flight output format:

- "Confirmed: ls/sv signatures in state.js at lines [N-M], examples: var v = ls('k', 'default'); sv('k', value);"
- "Confirmed: Firebase init pattern in integrations.js at line [N]"
- "Confirmed: Netlify proxy URL constant: [name] = '[url]' at integrations.js line [N]"
- "Confirmed: claude-proxy.js accepts JSON body matching Anthropic /v1/messages schema"
- "Confirmed: HQ load order is state -> integrations -> yumi-brain -> yumi-ui -> views -> app (book app adds arcs.js between yumi-brain and yumi-ui)"
- "Up to 3 questions if genuinely ambiguous: [questions]"
- "Pre-flight complete – proceeding to Sub-stage 1.1?" if all confirmed

=====

NON-GOALS – what Stage 1 must NOT do.

=====

- Do NOT write yumi-brain.js, arcs.js, yumi-ui.js, views.js, or app.js logic. Stage 1 creates stubs for these but does not implement them. Stage 2 implements yumi-brain. Later stages implement the rest.
- Do NOT write Firestore security rules. Those come at Stage 6 when the social layer ships.
- Do NOT modify any HQ file. Read-only reference only.
- Do NOT introduce const, let, arrow functions, classes, or template literals anywhere. Vanilla-JS conventions are non-negotiable.
- Do NOT add any styling beyond the bare minimum theme.css with CSS variable definitions. Full styling comes in Stage 2 and beyond.
- Do NOT implement book capture UI. Stage 3 owns that. Sub-stage 1.4 only verifies the fetch works in the console.
- Do NOT cache books, authors, or user data offline beyond what the service worker registers. Phase 1 testing does not need full offline caching.

=====

CONVENTIONS – locked, non-negotiable.

=====

Code style:

- var and function only – no const, let, arrow, class, template literals
- String concatenation only (no backticks for templating)
- CSS variables only (no hex literals in component code)
- localStorage via ls(k, d) / sv(k, v) wrappers – same signatures as HQ
- Fixed file load order: state -> integrations -> yumi-brain -> arcs -> yumi-ui -> views -> app

File paths (all paths absolute, all under /home/preston/praxis-app/):

- index.html
- manifest.json
- sw.js
- js/state.js
- js/integrations.js
- js/yumi-brain.js (stub only at Stage 1)
- js/arcs.js (stub only at Stage 1)
- js/yumi-ui.js (stub only at Stage 1)
- js/views.js (stub only at Stage 1)
- js/app.js (minimal init only at Stage 1)
- js/verify-engine.js (lifted from HQ verbatim)
- js/voice-input.js (lifted from HQ verbatim)
- assets/theme.css (CSS variable definitions only at Stage 1)
- netlify/functions/claude-proxy.js (lifted from HQ verbatim)
- docs/yumi-voice.md (already exists from Stage 0)

Brand position constraints (apply to every stage):

- Yumi never summarizes books
- Notebook is fully private, structurally enforced
- One Book Artifact per user per book – uniqueness constraint in data model
- Star ratings de-emphasized
- No follower counts as primary UI

=====

SUB-STAGES.

=====

Sub-stage 1.1 – Repo skeleton

Files to create:

- /home/preston/praxis-app/index.html
- /home/preston/praxis-app/manifest.json
- /home/preston/praxis-app/sw.js
- /home/preston/praxis-app/assets/theme.css (CSS variable definitions only)
- /home/preston/praxis-app/js/state.js (header comment + SCHEMA_VERSION constant + ls/sv wrappers, no schema yet)
- /home/preston/praxis-app/js/integrations.js (header comment + Netlify proxy URL constant only, no Firebase yet)
- /home/preston/praxis-app/js/yumi-brain.js (stub: header comment + empty IIFE)
- /home/preston/praxis-app/js/arcs.js (stub: header comment + empty IIFE)
- /home/preston/praxis-app/js/yumi-ui.js (stub: header comment + empty IIFE)
- /home/preston/praxis-app/js/views.js (stub: header comment + empty IIFE)
- /home/preston/praxis-app/js/app.js (minimal: header comment + DOMContentLoaded handler that console.logs "App init")

Files to copy verbatim from HQ:

- js/verify-engine.js
- js/voice-input.js
- netlify/functions/claude-proxy.js

What this sub-stage does:

- Establish the directory structure
- Establish the file load order in index.html
- Confirm that opening index.html in a browser logs "App init" to the console with no errors

Self-checks (all must PASS to proceed):

- All 13 expected files exist at expected paths (`ls` confirms each)
- index.html has 9 script tags in the correct fixed order: state -> integrations -> yumi-brain -> arcs -> yumi-ui -> views -> app, plus voice-input and verify-engine as utilities
- index.html links to /assets/theme.css
- index.html links to /manifest.json
- index.html registers /sw.js as the service worker
- Each new JS file is between 300 and 1500 bytes (small stubs, not empty, not bloated)
- No const/let/arrow/class/backtick appears in any file (grep returns 0)
- Opening index.html in a browser produces no console errors and logs "App init"

Verification commands:

- ls -la /home/preston/praxis-app/ /home/preston/praxis-app/js/ /home/preston/praxis-app/assets/ /home/preston/praxis-app/netlify/functions/
- grep -c "<script" /home/preston/praxis-app/index.html
- grep -E "const |let |=>|class |\"" /home/preston/praxis-app/js/*.js (must return zero matches)
- wc -c /home/preston/praxis-app/js/*.js
- node -c /home/preston/praxis-app/js/state.js (and same for each js file)

Expected byte ranges:

- index.html: 1500-2500 bytes
- state.js: 400-800 bytes (just constant + wrappers)
- integrations.js: 300-600 bytes
- yumi-brain.js, arcs.js, yumi-ui.js, views.js: 200-400 bytes each (stubs)
- app.js: 300-600 bytes
- theme.css: 1500-3000 bytes (CSS variables for fonts, colors, spacing)

If any check FAILS: show the failing check with actual evidence, stop, wait for instruction.

Sub-stage 1.2 – State schema and storage

Files to modify:

- /home/preston/praxis-app/js/state.js

What this sub-stage does:

- Add the data model schemas (as commented JSDoc-style headers, not enforced types – vanilla-JS doesn't have them)
- Add a `state` object with empty arrays/objects for the v1 collections: users, books, userBooks, notebooks (and stubs for collections that come later)
- Add `loadState()` that reads from localStorage via `ls()`
- Add `saveState()` that writes via `sv()`
- Add `state.SCHEMA\_VERSION = '1.0.0'` and a no-op migration function for now

Self-checks for Sub-stage 1.2:

- state.js parses without syntax errors (``node -c``)
- grep finds: `SCHEMA_VERSION`, `function loadState`, `function saveState`, `function migrate`
- grep finds the four v1 collection names: `users`, `books`, `userBooks`, `notebooks`
- File size is between 1200 and 2500 bytes
- Loading `index.html` in a browser, calling ``loadState()`` returns an object with the expected shape (test in console)
- No `const/let/arrow/class/backtick` appears (grep returns 0)

Verification commands:

- `grep -c "SCHEMA_VERSION\\|loadState\\|saveState\\|migrate" /home/preston/praxis-app/js/state.js`
- `grep -E "const |let |=>|class |\\`" /home/preston/praxis-app/js/state.js` (must return 0)
- `wc -c /home/preston/praxis-app/js/state.js`
- `node -c /home/preston/praxis-app/js/state.js`

Sub-stage 1.3 – Firebase + Google Auth

Files to modify:

- `/home/preston/praxis-app/js/integrations.js`
- `/home/preston/praxis-app/index.html` (add Firebase SDK script tags before `js/state.js`)

What this sub-stage does:

- Add Firebase SDK script tags to `index.html` (Auth + Firestore)
- Add Firebase config initialization in `integrations.js`
- Add ``signInWithGoogle()`` that triggers the popup auth flow and saves the user object to `localStorage` via ``sv()``
- Add ``signOut()`` that clears local auth state and Firebase auth
- Add ``getCurrentUser()`` that returns the current user from `localStorage` or null

PRECONDITION FOR THIS SUB-STAGE: Preston has provided a Firebase project configuration. If the agent does not have this, ask Preston for it as the one allowed question and STOP until provided. Do not invent placeholder Firebase keys.

Self-checks for Sub-stage 1.3:

- `integrations.js` parses without syntax errors
- grep finds: `signInWithGoogle`, `signOut`, `getCurrentUser`, `firebase.initializeApp`
- `index.html` includes Firebase SDK `<script>` tags before `js/state.js`
- After loading `index.html` in a browser, manually clicking a Sign In button (a temporary one Preston can click) triggers the Google popup
- After successful sign-in, ``getCurrentUser()`` in console returns the user object
- After refresh, ``getCurrentUser()`` still returns the same user (`localStorage` persistence works)
- No `const/let/arrow/class/backtick` (grep returns 0)

Verification commands:

- `grep -c "signInWithGoogle\\|signOut\\|getCurrentUser\\|initializeApp" /home/preston/praxis-app/js/integrations.js`
- `grep -E "const |let |=>|class |\\`" /home/preston/praxis-app/js/integrations.js` (must return 0)
- `node -c /home/preston/praxis-app/js/integrations.js`
- Manual test: sign in, refresh, verify persistence

Sub-stage 1.4 – Open Library + Google Books fetch

Files to modify:

- /home/preston/praxis-app/js/integrations.js

What this sub-stage does:

- Add `fetchBookByIsbn(isbn, callback)` that fetches from Open Library, falls back to Google Books, normalizes the response to a single shape: { isbn, title, author, coverUrl, publishYear, openLibraryWorkId }
- Pure read function. No write to state yet.
- Use callback-style for vanilla-JS compatibility (no Promises in component code; can use Promises internally if HQ does, otherwise use XMLHttpRequest)

Self-checks for Sub-stage 1.4:

- integrations.js parses without syntax errors
- grep finds: fetchBookByIsbn, openlibrary.org, googleapis.com/books
- Manual test in console: `fetchBookByIsbn('9780374533557', function(b) { console.log(b); })` returns a normalized book object for "Thinking, Fast and Slow" by Daniel Kahneman
- The returned object has all six fields populated (title, author, coverUrl, etc.)
- No const/let/arrow/class/backtick (grep returns 0)

Verification commands:

- `grep -c "fetchBookByIsbn\|openlibrary\|googleapis" /home/preston/praxis-app/js/integrations.js`
- `grep -E "const |let |=>|class |\`" /home/preston/praxis-app/js/integrations.js` (must return 0)
- `node -c /home/preston/praxis-app/js/integrations.js`
- Manual test: ISBN fetch returns valid book object

----- Sub-stage 1.5 – Service worker + manifest -----

Files to modify:

- /home/preston/praxis-app/sw.js
- /home/preston/praxis-app/manifest.json

What this sub-stage does:

- Implement basic service worker (cache app shell, network-first for API calls)
- Implement valid PWA manifest (name, short_name, icons placeholder, display: standalone, start_url, theme_color)
- CACHE_VERSION constant in sw.js, matching the pattern HQ uses
- App registers as installable PWA in Chrome/Safari

Self-checks for Sub-stage 1.5:

- sw.js parses without syntax errors
- manifest.json parses as valid JSON
- grep finds CACHE_VERSION in sw.js
- After loading index.html in Chrome, the install prompt appears (or DevTools > Application shows the app as installable)
- Service worker registers successfully (DevTools > Application > Service Workers)

Verification commands:

- `grep -c "CACHE_VERSION" /home/preston/praxis-app/sw.js`
- `node -c /home/preston/praxis-app/sw.js`
- `python3 -m json.tool /home/preston/praxis-app/manifest.json` (validates JSON)

=====

FINAL SUMMARY — at end of Sub-stage 1.5 only.

...

STAGE 1 COMPLETE

Files created: 13

Files modified across sub-stages: state.js, integrations.js, index.html, sw.js, manifest.json

Repo structure verified: yes

File load order verified: state -> integrations -> yumi-brain -> arcs -> yumi-ui -> views -> app
(utilities loaded separately)

Vanilla-JS conventions verified: no const/let/arrow/class/backticks across all files (grep
returns 0)

Firebase auth working: yes (manual sign-in tested)

Open Library fetch working: yes (ISBN test returned valid book)

PWA installable: yes

Brand constraints verified: no UI yet, but data model carries privacy distinction (notebook
collection separate from public collections)

Total bytes across all created files: [N]

Open questions for Preston: [list or "none"]

Recommended next stage: Stage 2 — Yumi brain (proxy wired, voice document loaded, first
conversation working)

...

NOTES FOR THE AGENT.

This stage is plumbing. The user-facing experience at end of Stage 1 is essentially: open the
app, sign in with Google, nothing visible happens, but the foundation is laid. That is correct.
Resist the temptation to add UI affordances "just to make it feel real" — those belong in later
stages. A clean Stage 1 makes every later stage faster.

If at any point you find yourself writing a const, let, or arrow function out of habit: stop,
rewrite, and treat it as a near-FAIL. The vanilla-JS conventions are not stylistic preferences.
They are the defining pattern of this codebase, and consistency with HQ matters because Preston
works across both.

If Firebase config is missing, ask once at the start of Sub-stage 1.3 and stop. Do not invent
fake config values, do not skip Firebase, do not implement a mock. Wait for the real config.

What you do during Stage 1: at the end of each sub-stage, read the CHECKPOINT REPORT
before you say GO. Spot-check the grep counts and byte sizes. If a sub-stage stalls because
Firebase config is missing, that is expected — pause, do the Firebase setup in Part Two, come
back with the config object.

Part Two

Setup steps — accounts, services, deploy infrastructure

Before you start anything

Four external services to set up. The order matters because Bookshop's approval is slow and Netlify needs something in the repo to deploy. The real order is: **Bookshop application first** (apply, then wait), **GitHub repo** with your existing markdown, **Firebase** setup, **Stage 0** (voice document), **Netlify** connected after the repo has the voice doc.

Quick checklist (in real order)

- Bookshop.org affiliate application submitted (do this first, approval is slow)
- GitHub account verified, repo created, existing markdown pushed
- Firebase project created, Authentication enabled with Google
- Firestore database created in test mode
- Firebase config object copied somewhere safe
- Stage 0 complete — yumi-voice.md committed to repo
- Netlify site connected to GitHub repo (after Stage 0)
- ANTHROPIC_API_KEY env var set on Netlify (existing key is fine)
- Netlify domain added to Firebase authorized domains
- Bookshop.org first storefront created (after approval)

If something stalls

Bookshop.org affiliate approval can take days or weeks. Apply early — ideally on day one. Firebase, GitHub, and Netlify are all near-instant.

Bookshop.org

Do this first. Approval is the slowest external dependency in the entire project — it can take a week. Apply now, then move on to GitHub and Firebase while you wait. The integration itself doesn't ship until Stage 8, but you need approval well before that.

Time

Thirty minutes for the application. Then wait for approval — usually a few business days.

Steps

- 1 Go to bookshop.org/info/affiliates and review the affiliate program. Standard commission is 10% on books sold through your links.
- 2 Click 'Become an Affiliate' or 'Apply'. You will need: a name, an email, a description of where you will be promoting books (the Praxis), an estimate of audience size (write your honest expected first-year reach).
- 3 Submit. You will get an email confirming the application. Approval typically takes 2 to 7 business days.
- 4 While you wait: think about what your first storefronts will be. The Bookshop model lets you create curated lists with their own URLs. For the Yumi app, every public Learning Arc auto-generates a storefront — but you will likely want a 'home' storefront with personal recommendations.
- 5 Once approved, log into bookshop.org and find **Affiliate Tools** → **Storefront** in the dashboard.
- 6 Create your first storefront. Name it 'Yumi reading lists' or use your brand name. Pick a slug for the URL (e.g. bookshop.org/shop/yumi-reading).
- 7 Note your **affiliate ID** from the dashboard. You will need it for the `integrations.js` bookshop functions in Stage 8.
- 8 Optional now, mandatory later: read Bookshop's API documentation. They have a partner program that allows affiliate links to be generated programmatically — Stage 8 will use this.

What you should have at the end

- Bookshop.org affiliate application submitted
- (After approval) First storefront created
- (After approval) Affiliate ID noted somewhere safe

Amazon Associates fallback: the spec calls for Amazon as a fallback for books Bookshop does not carry. Apply for Amazon Associates separately at affiliate-program.amazon.com when you reach Stage 8. Do not do it now — Amazon's approval requires a 'live website with content' which the app will not fully be until Stage 7 or 8.

GitHub

Where the source code lives. The repo can start with just your existing markdown files (spec, timeline, buildlog, the prompt files, this PDF). Stage 0 adds the voice document. Stage 1 adds the actual app code.

Time

Ten to fifteen minutes if you do not already have an account.

Steps

- 1 Go to github.com and create an account if you do not have one. Use your real name or a brand handle you will keep — repo URLs include the username.
- 2 Click the green 'New' button to create a new repository. Name it **praxis-app** or whatever you decide. Set it to Private for now. Do not add a README, .gitignore, or license — you will push those from your local folder.
- 3 On your computer, create the project folder: `mkdir ~/praxis-app` (or wherever you keep projects). Open a terminal in that folder.
- 4 Initialize git: `git init`
- 5 Create a basic .gitignore. At minimum exclude: `node_modules/`, `.env`, `.netlify/`, `.DS_Store`.
- 6 Connect to GitHub: `git remote add origin https://github.com/YOUR_USERNAME/praxis-app.git`
- 7 Make an initial commit: `git add . && git commit -m 'initial commit'`
- 8 Push to GitHub: `git push -u origin main` (you may need to authenticate the first time).

What you should have at the end

- A private GitHub repo at `github.com/USERNAME/praxis-app`
- A local project folder connected to that repo
- At least one commit pushed

Firestore

Authentication and database. Sub-stage 1.3 of Stage 1 will block until you have the Firebase config in hand, so do this before starting Stage 1.

Time

Fifteen to twenty minutes.

Steps

- 1 Go to `console.firebase.google.com`. Sign in with your existing Google account (the same one you use for HQ and other projects is fine).
- 2 Click 'Add project'. Name it **praxis** or **praxis-app**. Disable Google Analytics for now — you can add it later if needed.
- 3 When the project loads, click **Build** → **Authentication** in the left sidebar. Click 'Get started'.
- 4 On the Sign-in method tab, click **Google**. Toggle Enable. Set your support email. Save.
- 5 Back in the sidebar, click **Build** → **Firestore Database**. Click 'Create database'. Choose **Start in test mode**. Pick a region close to you. Click Enable.
- 6 Get the config. Click the gear icon next to 'Project Overview' → **Project settings**.
- 7 Scroll down to 'Your apps'. Click the **web icon** (`</>`) to register a web app. Nickname: 'Praxis Web'. Do not enable Firebase Hosting (Netlify will host).
- 8 After registering, Firebase shows you a config object. **Copy this entire object somewhere safe**. It looks like:

```
const firebaseConfig = {
  apiKey: "AIza...",
  authDomain: "praxis-app.firebaseio.com",
  projectId: "praxis-app",
  storageBucket: "praxis-app.appspot.com",
  messagingSenderId: "...",
  appId: "1:...",
};
```

- 9 In Authentication → Settings → Authorized domains, ensure your Netlify domain (you will know it after Netlify setup) and `localhost` are both listed.

What you should have at the end

- Firebase project created

- Authentication enabled with Google provider
- Firestore database in test mode
- Web app registered in Project Settings
- Firebase config object saved somewhere safe

Important: Test mode for Firestore lets anyone read and write your database. This is fine for solo development, but you must add real security rules before any other user touches the app — Stage 6 covers this. Do not forget.

Netlify

Hosting plus serverless functions. **Do this after Stage 0, not before.** Once Stage 0 produces docs/yumi-voice.md and you push it to the repo, the repo has at least one real artifact. Then Netlify deploys something meaningful (your README and your voice doc) instead of an empty repo.

Time

Ten minutes for the site, plus a few more for environment variables.

Steps

- 1 Go to netlify.com and sign up with your GitHub account. Authorize Netlify to access your repos.
- 2 Click 'Add new site' → 'Import an existing project'. Choose GitHub. Find and select your **praxis-app** repo.
- 3 Build settings: leave the build command empty for now (the app is static HTML/JS, no build step). Publish directory: leave as the repo root or set to `./`. Click Deploy.
- 4 Netlify gives the site a random subdomain like `shimmering-cake-12345.netlify.app`. You can rename it under Site settings → Site information → Change site name.
- 5 Click **Site settings** → **Environment variables**. Add a new variable: name `ANTHROPIC_API_KEY`, value is your existing Anthropic API key (the same key you use for HQ — no need to create a new one). This is what `claude-proxy.js` will use.
- 6 Confirm Netlify Functions are enabled. Under Site settings → Functions, the directory should be `netlify/functions`. Stage 1.1 creates this folder structure.
- 7 Go back to Firebase Authentication → Settings → Authorized domains. Add your Netlify domain so Google sign-in works on the deployed site.

What you should have at the end

- Netlify site connected to your GitHub repo
- Netlify domain noted (e.g. `praxis-app.netlify.app`)
- `ANTHROPIC_API_KEY` environment variable set
- Netlify Functions directory configured
- Netlify domain added to Firebase authorized domains

Note on the proxy function: the file `netlify/functions/claude-proxy.js` will be created in Stage 1.1 (lifted from HQ). Once you push that file to GitHub, Netlify auto-deploys and the function is live at `/.netlify/functions/claude-proxy`. Stage 2 wires Yumi to it.

What to save, log, and verify

After every stage completes, do these three things before moving on. They take fifteen minutes and prevent context loss across sessions.

1. Save deliverables

Whatever the stage produced, commit to GitHub. `git add . && git commit -m 'Stage N complete' && git push`. The git history becomes the audit trail of what was built when.

2. Log the session in book-app-buildlog.md

Open your buildlog. Append a new entry following the template. Capture: what you built, what you decided, what surprised you, what to remember next time. Three or four sentences each. The point is the ritual.

3. Update the living document

Open the HTML living document in a Claude chat (re-upload the file). Tell Claude what changed. Claude updates the status card, the stage table, and appends a dated entry to the update log. Save the new HTML over the old one.

Three questions to ask yourself between stages

Did the stage actually achieve its goal? The goal sentence at the top of each prompt is the test. If you cannot honestly say yes, the stage is not done. Do not move on.

Did the agent follow the conventions? Spot-check: open one or two of the new files. Are they var/function only? CSS variables only? localStorage via ls/sv? If you find a const slipped in, fix it before the next stage — drift compounds.

What is the one thing you would want to remember six months from now about this stage? Write that down in the buildlog. Future-you will thank you.

The whole point of this companion document: the build is long and solo. Drift is the enemy. The discipline above is what keeps Stage 9 the same shape as Stage 0. Trust the process more than the urge to skip steps.

End of document.